**Aim**: This report is to determine the optimal threshold for each edge detection operator (Sobel, Prewitt and Laplacian) applied to HEp-2 cell images. For advanced edge detectors, the Canny operator is being used in Task 2 to identify edge points with precision. Additionally, the report includes performing a Hough Transform in Task 3 to detect possible lines, using the edges detected by the Canny method as a precursor to line detection.

**TASK 1**: To evaluate how well the Sobel, Prewitt, and Laplacian edge detectors identify Hep-2 cell boundaries.

**Method Overview**: All images were first converted from RGB to grayscale (using rgb2gray) and normalised to the range [0,1] to provide a consistent intensity scale across all datasets. Because fluorescence microscopy introduces Gaussian-like noise and low-contrast edges, Gaussian smoothing ($\sigma = 1$) was applied to suppress high-frequency noise, followed by CLAHE to enhance local boundary contrast. To visualise images are inverted to look like ground truth. These preprocessing steps were implemented using standard scikit-image functions. Sobel, Prewitt and Laplacian operators were implemented manually using 3×3 convolution kernels and scipy.signal.convolve2d. Sobel and Prewitt compute horizontal and vertical gradients, which were combined using the magnitude. Sobel uses larger central weights (−2, 0, 2), producing stronger but thicker edges, while Prewitt uses uniform weights that generate smoother gradients. The Laplacian applies a second-order derivative and is more sensitive to noise. All edge-response images were renormalised to [0,1] so that threshold values represent a fixed proportion of the maximum gradient strength, allowing thresholds to be applied consistently across detectors. Binary edge maps were generated using thresholds T between [0,1].
To quantitatively assess performance, each thresholded result was compared with the ground truth using:

- IoU (Jaccard Index) – the overlap between predicted and true edges.
- Dice coefficient (F1-score) – twice the intersection over the total number of edge pixels.
- Pixel accuracy – fraction of correctly classified pixels.

**Results**

**The Sobel Operator** computes image gradients using weighted horizontal and vertical kernels, producing strong but relatively thick edges that are sensitive to noise. In the HEp-2 cell images, Sobel detected most outer boundaries, but internal texture variations caused false positives and broader contours. Across the three images (Figure 1), optimal thresholds fell between T = 0.20 and 0.30, yielding IoU values of approximately 0.32–0.41 and Dice scores of 0.48–0.58. The best thresholds for: Image 43590 AM => T = 0.20 (IoU = 0.4068), Image 10905 JL => T = 0.25 (IoU = 0.3250), and Image 9343 AM => T = 0.30 (IoU = 0.3548).

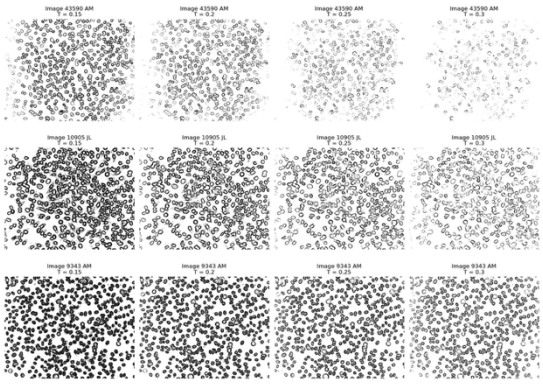


Figure 1. Sobel Operator on 3 Images

**The Prewitt Operator** uses uniform derivative masks. It produced the cleanest and thinnest boundaries of the three detectors. The use of uniform gradient kernels made it less sensitive to noise and internal intensity variations, resulting in smoother and more continuous edges that closely resembled the ground truth. Across the three images (Figure 2), the best thresholds were T = 0.20 for Image 43590 AM (IoU = 0.4086), T = 0.25 for Image 10905 JL (IoU = 0.3236), and T = 0.35 for Image 9343 AM (IoU = 0.3612). These values are comparable to or slightly higher than those obtained with Sobel, while the visual results show noticeably less noise and more accurate contour localisation.

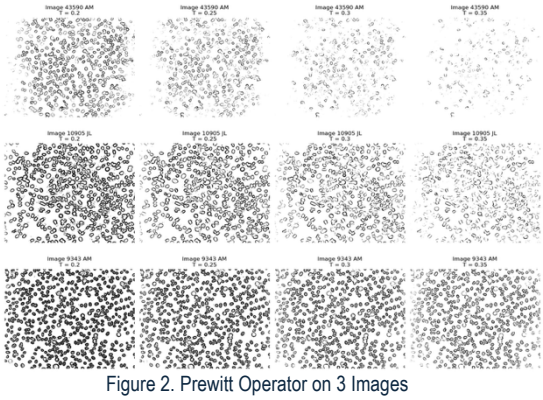


Figure 2. Prewitt Operator on 3 Images

**The Laplacian Operator** uses a second-order operator which responds to rapid intensity changes but is highly sensitive to noise. It performed poorly on all three images due to its reliance on second-order derivatives, which greatly amplify noise and small intensity fluctuations typical of fluorescence microscopy. Even after Gaussian smoothing, it produced fragmented, noisy, and discontinuous edges that showed weak correspondence with the ground truth. The best thresholds yielded very low IoU values: T = 0.10 for Image 43590 AM (IoU = 0.0696), T = 0.10 for Image 10905 JL (IoU = 0.0577), and T = 0.10 for Image 9343 AM (IoU = 0.1068)

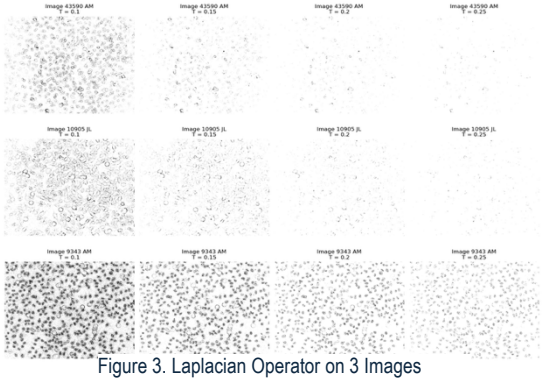


Figure 3. Laplacian Operator on 3 Images

confirming that the operator fails to isolate true cell boundaries reliably. Visually (Figure 3), the Laplacian maps were dominated by noise and edge fragmentation, making it unsuitable for these images compared with the first-order Sobel and Prewitt methods.

**Conclusion**: Overall, the results show that the preprocessing steps: grayscale conversion, normalisation, inversion, Gaussian smoothing and CLAHE were essential for improving edge visibility and providing consistent input for all detectors. Although automatic thresholding methods such as Otsu's algorithm could have been used, selecting thresholds based on IoU with the ground truth provided a more reliable and objective way to optimise edge quality for this dataset. Visual inspection and quantitative evaluation (IoU, Dice and pixel accuracy) indicate that Prewitt performed best, producing thin, smooth and coherent boundaries with the highest or comparable scores. Sobel detected the main cell outlines but generated thicker edges and more false positives due to its sensitivity to internal texture. The Laplacian operator performed poorly: its second-order derivative amplified noise and produced fragmented edges despite smoothing, resulting in very low IoU values. Overall, Prewitt is the most reliable classical edge detector for HEp-2 cell boundaries.

**TASK 2**: To apply Canny edge detector to the same images and compare its performance with the methods from Task1.

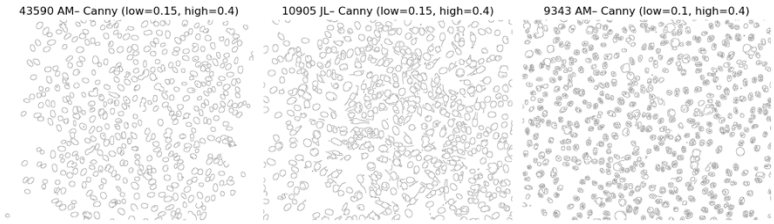
**Canny Edge Detector** is a multi-stage edge detection algorithm. The image is first smoothed with a Gaussian filter to reduce noise, then the gradient magnitude and direction are computed (similar to Sobel). Non-maximum suppression thins the edges by keeping only local maxima along the gradient direction. Finally, double thresholding and hysteresis are applied: strong edges above a high threshold are kept, weak edges above a lower threshold are kept only if they are connected to strong edges. This produces thin, continuous edges while suppressing many spurious responses.

**Method Overview**: Although Canny performs its own Gaussian smoothing, I applied the same preprocessing pipeline as Task 1 (grayscale conversion, normalisation, inversion, external Gaussian filtering, and CLAHE). This ensured fair comparison across detectors and improved Canny's performance by enhancing contrast and suppressing larger-scale noise that its internal filter does not remove. After preprocessing, Canny was applied using a small grid of low and high threshold values. For each (low, high) pair, an edge map was generated and compared with the ground-truth edges using the IoU score. The threshold pair that produced the highest IoU for each image was selected as the optimal setting. This grid-search approach ensured that the chosen thresholds balanced edge continuity and noise suppression based on actual performance rather than manual tuning. The implementation used skimage.feature.canny.

**Results**

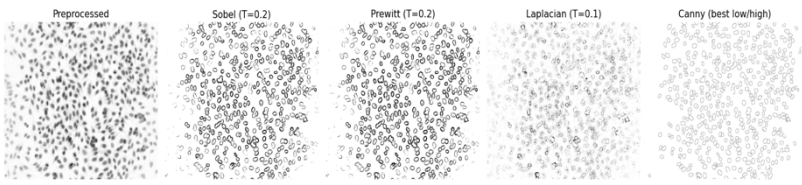
Canny produced thin, well-localised and continuous edges on all three HEp-2 images. Its non-maximum suppression avoided thick or blurred contours, giving visually cleaner boundaries than Sobel and especially Laplacian. The best threshold pairs identified by the IoU-based grid search were:

- 43590 AM: low = 0.15, high = 0.40
- 10905 JL: low = 0.15, high = 0.40
- 9343 AM: low = 0.10, high = 0.40




A comparison of IoU scores across all detectors is shown below:

| Image | Sobel IoU | Prewitt IoU | Laplacian IoU | Canny IoU |
|---|---|---|---|---|
| 43590 AM | 0.4068 | 0.4086 | 0.0827 | 0.2421 |
| 10905 JL | 0.3250 | 0.3236 | 0.0412 | 0.1454 |
| 9343 AM | 0.3548 | 0.3612 | 0.1068 | 0.2055 |




Prewitt achieves the highest IoU values, slightly outperforming Sobel, while Laplacian performs very poorly due to noise amplification. Canny produces the visually cleanest edges, but its IoU is lower because its contours are much thinner than the thicker hand-drawn ground-truth edges. The threshold grid confirmed that higher low-threshold values reduce noise but may remove weak boundaries, while lower thresholds preserve continuity at the cost of including more responses.

**Conclusion**: The Canny detector produces the visually cleanest and most continuous edges among all methods tested. Its multi-stage design: Gaussian smoothing, non-maximum suppression, and hysteresis effectively removed noise and

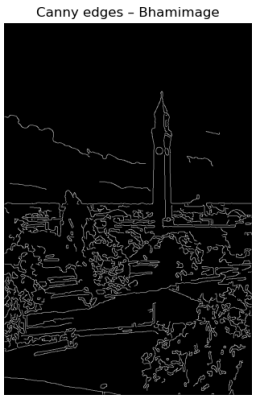
preserved true boundaries, giving more coherent contours than Sobel, and especially Laplacian. However, its IoU scores were lower due to thinner edges than those in the ground truth. Canny still offered superior practical delineation of the cell shapes. Overall, Canny proved to be the most reliable and visually accurate detector for these HEp-2 images.

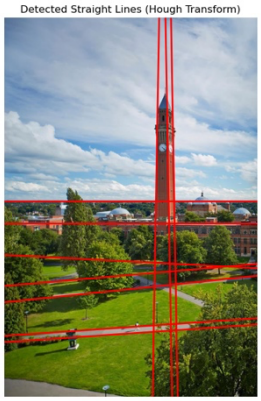
**TASK 3**: To extract edges using Canny and apply Hough Transform to detect and assess prominent lines in the image.

**Method overview**: The "Bhamimage.jpeg" image was first converted to grayscale and processed using the Canny edge detector (skimage.feature.canny) to obtain a binary edge map. The straight-line Hough Transform was then applied using hough_line from skimage.transform, evaluating line candidates across angles from $-\frac{\pi}{2}$ to $\frac{\pi}{2}$. Peaks in the Hough accumulator were extracted with hough_line_peaks, giving the most prominent line parameters. These detected lines were finally plotted back onto the original image. Two main outputs were produced: the Canny edge map and the final image with straight lines overlaid.

**Result**

**Canny detector** is applied to the image and produced a clear edge map containing the major structural outlines of the buildings, Old Joe (the clock tower) and the horizon. Canny successfully preserved strong edges while suppressing noise, providing thin, well-localised contours suitable for line detection. This edge map served as the input to the straight-line Hough Transform.




**Hough Transform:** The Hough accumulator constructed from the edge points showed several strong peaks, indicating dominant straight structures at specific orientations. Extracting these peaks with hough_line_peaks yielded a small set of prominent lines. When these lines were projected back onto the original image, the strongest vertical and horizontal architectural lines were correctly identified for example, the edges of Old Joe and the building façades.

**Effectiveness and discussion**: However, the method also produced several false positives. Strong red lines appeared across regions such as grass, trees and parts of the sky where no real straight edges exist. These occurred because textured areas produced many small Canny edge fragments that happened to align, leading the Hough Transform to incorrectly vote for long straight lines. There were also missing lines. Some genuine but weaker edges such as thin roof boundaries or lines partially hidden by trees were not detected. These edges were either too faint for Canny to retain or too short and fragmented to accumulate enough votes in Hough space.

**Conclusion**: The Canny and Hough Transform approach successfully identified the major straight structural lines in the scene, particularly those from prominent architectural features such as Old Joe and building façades. While effective for strong, continuous edges, the method showed limitations: several false positives appeared in textured natural regions and weaker or partially occluded lines were missed. Overall, the straight-line Hough Transform is reliable for detecting dominant linear features but is sensitive to noise and edge discontinuities, making it less effective for complex natural textures and faint structures.

**References**


- Scikit image:Van der Walt, S. et al. (2014) scikit-image: Image processing in Python. PeerJ, 2, p. e453.
- Scipy: Virtanen, P. et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, pp. 261–272.


**Note**: All image files were loaded using relative file paths (e.g., Data_30213/cells/43590 AM.bmp). The images provided with the assignment must be placed in the same folder structure for the notebook to run correctly.